

```
def suma_gauss(n):  
    y = (n*(n+1))/2  
    return y  
  
suma_gauss(100)
```

5050.0

```
#ciclo for para la misma suma de gauss  
  
S = 0  
for a in range(1,101):  
    S = S+a  
  
print(S)
```

5050

Comienza a programar o generar con IA.

```
majo = list(range(1,11))  
print (majo)
```

[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

```
# Definimos una lista de numeros  
numeros = [10 , 20 , 30 , 40 , 50]  
  
# Inicializamos la variable suma en cero  
suma_total = 0  
# El ciclo for recorre cada elemento de la lista ' numeros '  
for numero in numeros :  
  
# En cada iteracion , ' numero ' es el valor actual de la lista  
    suma_total = suma_total + numero  
  
# Mostramos el resultado  
print ( " La suma de los numeros es : " , suma_total )
```

La suma de los numeros es : 150

El Ciclo for en Estructuras de Datos

Curso de Programación Básica

9 de septiembre de 2026

Índice

1. Introducción	2
2. El Ciclo for con Listas	3
2.1. ¿Qué es una Lista?	3
2.2. Uso del Ciclo <code>for</code>	3
2.3. Ejemplo: Suma de Elementos	3
3. El Ciclo for con Arreglos (usando <code>range()</code>)	4
3.1. ¿Qué es un Arreglo en este Contexto?	4
3.2. Uso del Ciclo <code>for</code>	4
3.3. Ejemplo: Tabla de Multiplicar	4
4. El Ciclo for con Matrices	5
4.1. ¿Qué es una Matriz?	5
4.2. Uso del Ciclo <code>for</code> Anidado	5
4.3. Ejemplo: Recorrer e Imprimir una Matriz	5
5. El Ciclo for con Tuplas	7
5.1. ¿Qué es una Tupla?	7
5.2. Uso del Ciclo <code>for</code>	7
5.3. Ejemplo: Recorrer Días de la Semana	7
6. Conclusión	7

1. Introducción

El ciclo `for` es una de las estructuras de control más fundamentales y poderosas en programación. Su propósito principal es **iterar** sobre una secuencia de elementos, permitiéndonos ejecutar un bloque de código para cada uno de ellos.

En Python, el ciclo `for` es extremadamente versátil y puede iterar sobre cualquier objeto que sea *iterable*, como listas, tuplas, cadenas de texto, diccionarios y, con la ayuda de funciones como `range()`, también sobre arreglos numéricos.

En este documento, exploraremos el uso del ciclo `for` con cuatro tipos de estructuras de datos fundamentales:

- **Listas:** Colecciones mutables y ordenadas.
- **Arreglos:** Nos referiremos a ellos como secuencias numéricas creadas con `range()`.
- **Matrices:** Listas anidadas que representan datos bidimensionales.
- **Tuplas:** Colecciones inmutables y ordenadas.

Cada sección incluirá una explicación teórica y un ejemplo sencillo para una comprensión clara del concepto.

2. El Ciclo for con Listas

2.1. ¿Qué es una Lista?

Una lista en Python es una colección de elementos que es **ordenada** y **modificable** (mutable). Esto significa que podemos agregar, eliminar o cambiar sus elementos después de creada. Se definen usando corchetes `[]`.

2.2. Uso del Ciclo for

La sintaxis es directa: `for elemento in lista:`. En cada iteración, la variable `elemento` tomará el valor del siguiente ítem en la lista.

2.3. Ejemplo: Suma de Elementos

El siguiente ejemplo muestra cómo usar un ciclo `for` para calcular la suma de todos los números en una lista.

```
1 # Definimos una lista de numeros
2 numeros = [10, 20, 30, 40, 50]
3
4 # Inicializamos la variable suma en cero
5 suma_total = 0
6
7 # El ciclo for recorre cada elemento de la lista 'numeros'
8 for numero in numeros:
9     # En cada iteracion, 'numero' es el valor actual de la lista
10    suma_total = suma_total + numero
11
12 # Mostramos el resultado
13 print("La suma de los numeros es:", suma_total)
```

Listing 1: Ejemplo de `for` con una lista para sumar elementos.

Explicación del Código

1. Se crea la lista `numeros` con cinco valores enteros.
2. La variable `suma_total` se inicializa en 0.
3. El ciclo `for numero in numeros:` itera sobre la lista.
4. En la primera iteración, `numero = 10`, y `suma_total` pasa a ser 10.
5. En la segunda, `numero = 20`, y `suma_total` pasa a ser 30 (10+20).
6. Este proceso se repite hasta el último elemento, resultando en una suma total de 150.

3. El Ciclo for con Arreglos (usando range())

3.1. ¿Qué es un Arreglo en este Contexto?

En muchos lenguajes, un arreglo es una estructura de tamaño fijo. En Python, cuando hablamos de arreglos en un contexto básico, a menudo nos referimos a una secuencia de números generada por la función `range()`. `range()` no crea una lista, sino un objeto iterable que genera números sobre la marcha, lo que es muy eficiente en memoria.

3.2. Uso del Ciclo for

El ciclo `for` puede iterar directamente sobre un objeto `range`. Esto es especialmente útil cuando necesitas ejecutar un bloque de código un número específico de veces o cuando necesitas los índices para acceder a otros elementos.

3.3. Ejemplo: Tabla de Multiplicar

Este ejemplo utiliza `range()` para generar los números del 1 al 10 y calcular la tabla de multiplicar del número 5.

```
1 # Definimos el numero base
2 numero_base = 5
3
4 # range(1, 11) genera los numeros 1, 2, 3, ..., 10
5 for i in range(1, 11):
6     # i toma el valor de 1 a 10 en cada iteracion
7     resultado = numero_base * i
8     print(f"{numero_base} x {i} = {resultado}")
```

Listing 2: Ejemplo de `for` con `range()` para una tabla de multiplicar.

Explicación del Código

1. La función `range(1, 11)` genera una secuencia de números desde 1 hasta 10 (el límite superior es exclusivo).
2. El ciclo `for` itera sobre esta secuencia. En la primera iteración, `i = 1`; en la segunda, `i = 2`, y así sucesivamente.
3. Para cada valor de `i`, se calcula el producto y se imprime la ecuación, generando así la tabla de multiplicar completa.

4. El Ciclo for con Matrices

4.1. ¿Qué es una Matriz?

Una matriz es una estructura de datos bidimensional que organiza la información en filas y columnas. En Python, la forma más común de representar una matriz es mediante una **lista de listas**. Cada lista interna representa una fila de la matriz.

4.2. Uso del Ciclo for Anidado

Para recorrer una matriz, necesitamos un ciclo `for` anidado:

- El **primer ciclo** itera sobre cada fila (cada lista interna).
- El **segundo ciclo** (anidado) itera sobre cada elemento dentro de esa fila.

4.3. Ejemplo: Recorrer e Imprimir una Matriz

Este ejemplo crea una matriz de 3x3 y usa ciclos `for` anidados para imprimir cada uno de sus elementos.

```
1 # Definimos una matriz 3x3 (lista de listas)
2 matriz = [
3     [1, 2, 3],
4     [4, 5, 6],
5     [7, 8, 9]
6 ]
7
8 # El primer ciclo 'for' recorre cada fila de la matriz
9 for fila in matriz:
10     # El segundo ciclo 'for' recorre cada elemento dentro de la fila
    actual
11     for elemento in fila:
12         # Imprimimos el elemento sin salto de línea, con un espacio
13         print(elemento, end=" ")
14     # Al terminar una fila, hacemos un salto de línea para el formato
15     print() # Salto de línea para separar filas
```

Listing 3: Ejemplo de `for` anidado para recorrer una matriz.

Explicación del Código

1. La variable `matriz` es una lista que contiene tres listas, cada una con tres números.
2. El ciclo exterior: `for fila in matriz`. En la primera iteración, `fila = [1, 2, 3]`.
3. El ciclo interior: `for elemento in fila`. En la primera iteración del ciclo exterior, `elemento` tomará los valores 1, 2 y 3, imprimiéndolos en una línea.
4. Una vez que el ciclo interior termina para la primera fila, el código ejecuta `print()` para un salto de línea y pasa a la siguiente fila (`fila = [4, 5, 6]`).

5. Este proceso se repite hasta que se han recorrido todas las filas y elementos de la matriz.

5. El Ciclo for con Tuplas

5.1. ¿Qué es una Tupla?

Una tupla es muy similar a una lista, pero con una diferencia crucial: es **immutable**. Una vez que se crea una tupla, no se pueden modificar, agregar ni eliminar elementos. Se definen usando paréntesis (). Se usan para almacenar datos que no deben cambiar, como los días de la semana o las coordenadas de un punto.

5.2. Uso del Ciclo for

Iterar sobre una tupla con un ciclo `for` es idéntico a hacerlo con una lista, ya que ambas son secuencias iterables.

5.3. Ejemplo: Recorrer Días de la Semana

Este ejemplo muestra cómo usar un ciclo `for` para recorrer una tupla que contiene los nombres de los días de la semana.

```
1 # Definimos una tupla con los días de la semana
2 dias_semana = ("Lunes", "Martes", "Miercoles", "Jueves", "Viernes", "
   Sabado", "Domingo")
3
4 # Recorremos la tupla e imprimimos cada día
5 print("Dias de la semana:")
6 for dia in dias_semana:
7     print(f"- {dia}")
```

Listing 4: Ejemplo de `for` con una tupla.

Explicación del Código

1. Se define la tupla `dias_semana` con los siete días.
2. El ciclo `for dia in dias_semana:` funciona de manera idéntica a como lo haría con una lista.
3. En cada iteración, la variable `dia` toma el valor del siguiente elemento de la tupla: primero "Lunes", luego "Martes", y así sucesivamente.
4. Cada valor se imprime en un formato de lista con guiones.

6. Conclusión

El ciclo `for` es una herramienta indispensable para el manejo de estructuras de datos. Como hemos visto, su sintaxis es consistente y elegante, permitiendo iterar sobre listas, secuencias numéricas, matrices anidadas y tuplas con la misma lógica. Dominar este ciclo es un paso crucial para escribir código eficiente y legible en Python y en la mayoría de los lenguajes de programación modernos.

Propiedades de la Sumatoria para la Integral Definida Implementación en Python

Documento Educativo

September 9, 2026

Abstract

Este documento explora la conexión fundamental entre las sumatorias y la integral definida a través de las sumas de Riemann. Presentamos las propiedades algebraicas de la sumatoria, su aplicación en la aproximación de integrales definidas, y dos ejemplos prácticos implementados en Python utilizando métodos numéricos.

1 Introducción

La integral definida de una función $f(x)$ en el intervalo $[a, b]$ se define como el límite de una suma de Riemann:

$$\int_a^b f(x) dx = \lim_{n \rightarrow \infty} \sum_{i=1}^n f(x_i^*) \Delta x$$

donde $\Delta x = \frac{b-a}{n}$ y x_i^* es un punto en el i -ésimo subintervalo.

Las propiedades de la sumatoria son herramientas esenciales para:

- Calcular límites de sumas de Riemann
- Simplificar expresiones complejas
- Implementar algoritmos numéricos de integración

2 Propiedades Fundamentales de la Sumatoria

2.1 Propiedades Algebraicas

1. Linealidad:

$$\begin{aligned} \sum_{i=1}^n (a_i + b_i) &= \sum_{i=1}^n a_i + \sum_{i=1}^n b_i \\ \sum_{i=1}^n c \cdot a_i &= c \sum_{i=1}^n a_i \end{aligned}$$

2. Suma de constantes:

$$\sum_{i=1}^n c = n \cdot c$$

3. Sumas notables:

$$\begin{aligned} \sum_{i=1}^n i &= \frac{n(n+1)}{2} \\ \sum_{i=1}^n i^2 &= \frac{n(n+1)(2n+1)}{6} \\ \sum_{i=1}^n i^3 &= \left(\frac{n(n+1)}{2} \right)^2 \end{aligned}$$

3 Métodos Numéricos de Integración

3.1 Suma de Riemann por la Izquierda

$$\int_a^b f(x) dx \approx \sum_{i=0}^{n-1} f(a + i\Delta x) \Delta x$$

3.2 Suma de Riemann por la Derecha

$$\int_a^b f(x) dx \approx \sum_{i=1}^n f(a + i\Delta x) \Delta x$$

3.3 Regla del Punto Medio

$$\int_a^b f(x) dx \approx \sum_{i=0}^{n-1} f\left(a + \left(i + \frac{1}{2}\right) \Delta x\right) \Delta x$$

3.4 Regla del Trapecio

$$\int_a^b f(x) dx \approx \frac{\Delta x}{2} \left[f(a) + 2 \sum_{i=1}^{n-1} f(a + i\Delta x) + f(b) \right]$$

4 Implementación en Python

4.1 Funciones Base

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def riemann_left(f, a, b, n):
5     """Suma de Riemann por la izquierda"""
6     dx = (b - a) / n
7     x = np.linspace(a, b - dx, n)
8     return np.sum(f(x)) * dx
9
10 def riemann_right(f, a, b, n):
11     """Suma de Riemann por la derecha"""
12     dx = (b - a) / n
13     x = np.linspace(a + dx, b, n)
14     return np.sum(f(x)) * dx
15
16 def riemann_midpoint(f, a, b, n):
17     """Regla del punto medio"""
18     dx = (b - a) / n
19     x = np.linspace(a + dx/2, b - dx/2, n)
20     return np.sum(f(x)) * dx
21
22 def trapezoidal(f, a, b, n):
23     """Regla del trapecio"""
24     dx = (b - a) / n
25     x = np.linspace(a, b, n + 1)
26     y = f(x)
27     return dx * (0.5 * y[0] + np.sum(y[1:-1]) + 0.5 * y[-1])
28
29 def simpson(f, a, b, n):
30     """Regla de Simpson (n debe ser par)"""
31     if n % 2 != 0:
32         raise ValueError("n debe ser par para la regla de Simpson")
33     dx = (b - a) / n
34     x = np.linspace(a, b, n + 1)
35     y = f(x)
36     return (dx/3) * (y[0] + y[-1] + 4*np.sum(y[1:-1:2]) + 2*np.sum(y[2:-1:2]))
```

Listing 1: Funciones para métodos de integración numérica

4.2 Función para Visualización

```
1 def plot_riemann(f, a, b, n, method='left'):  
2     """Visualiza la aproximación de Riemann"""  
3     dx = (b - a) / n  
4     x_plot = np.linspace(a, b, 1000)  
5     y_plot = f(x_plot)  
6  
7     if method == 'left':  
8         x_rect = np.linspace(a, b - dx, n)  
9     elif method == 'right':  
10        x_rect = np.linspace(a + dx, b, n)  
11    else: # midpoint  
12        x_rect = np.linspace(a + dx/2, b - dx/2, n)  
13  
14    y_rect = f(x_rect)  
15  
16    plt.figure(figsize=(10, 6))  
17    plt.plot(x_plot, y_plot, 'b-', linewidth=2, label='f(x)')  
18    plt.bar(x_rect, y_rect, width=dx, alpha=0.3, align='edge',  
19           edgecolor='red', label=f'Riemann {method}')  
20    plt.xlabel('x')  
21    plt.ylabel('f(x)')  
22    plt.title(f'M todo de Riemann ({method}) con n = {n}')  
23    plt.legend()  
24    plt.grid(True, alpha=0.3)  
25    plt.show()
```

Listing 2: Visualización de la aproximación

5 Ejemplos Prácticos

5.1 Ejemplo 1: Integral de $f(x) = x^2$ en $[0, 2]$

Problema: Aproximar $\int_0^2 x^2 dx$ usando diferentes métodos.

Solución analítica:

$$\int_0^2 x^2 dx = \left[\frac{x^3}{3} \right]_0^2 = \frac{8}{3} \approx 2.6667$$

```
1 # Definir la función  
2 def f1(x):  
3     return x**2  
4  
5 # Parámetros  
6 a, b = 0, 2  
7 n = 10 # Número de subintervalos  
8  
9 # Calcular aproximaciones  
10 left = riemann_left(f1, a, b, n)  
11 right = riemann_right(f1, a, b, n)  
12 mid = riemann_midpoint(f1, a, b, n)  
13 trap = trapezoidal(f1, a, b, n)  
14 simp = simpson(f1, a, b, n) # n debe ser par  
15  
16 # Valor exacto  
17 exact = 8/3  
18  
19 # Mostrar resultados  
20 print("Resultados para          x   dx:")  
21 print(f" Valor exacto:          {exact:.6f}")  
22 print(f" Riemann izquierda:      {left:.6f} (error: {abs(left-exact):.6f})")  
23 print(f" Riemann derecha:        {right:.6f} (error: {abs(right-exact):.6f})")  
24 print(f" Punto medio:           {mid:.6f} (error: {abs(mid-exact):.6f})")  
25 print(f" Trapecio:              {trap:.6f} (error: {abs(trap-exact):.6f})")  
26 print(f" Simpson:               {simp:.6f} (error: {abs(simp-exact):.6f})")  
27  
28 # Visualizar  
29 plot_riemann(f1, a, b, n, 'left')
```

Listing 3: Código para el Ejemplo 1

5.2 Ejemplo 2: Integral de $f(x) = \sin(x)$ en $[0, \pi]$

Problema: Aproximar $\int_0^\pi \sin(x) dx$ y estudiar la convergencia.

Solución analítica:

$$\int_0^\pi \sin(x) dx = [-\cos(x)]_0^\pi = 2$$

```
1 # Definir la función
2 def f2(x):
3     return np.sin(x)
4
5 # Parámetros
6 a, b = 0, np.pi
7 n_values = [5, 10, 20, 50, 100]
8
9 # Calcular errores para diferentes n
10 errors = {'left': [], 'right': [], 'mid': [], 'trap': [], 'simp': []}
11 exact = 2
12
13 for n in n_values:
14     left = riemann_left(f2, a, b, n)
15     right = riemann_right(f2, a, b, n)
16     mid = riemann_midpoint(f2, a, b, n)
17     trap = trapezoidal(f2, a, b, n)
18     simp = simpson(f2, a, b, n) if n % 2 == 0 else None
19
20     errors['left'].append(abs(left - exact))
21     errors['right'].append(abs(right - exact))
22     errors['mid'].append(abs(mid - exact))
23     errors['trap'].append(abs(trap - exact))
24     if simp is not None:
25         errors['simp'].append(abs(simp - exact))
26     else:
27         errors['simp'].append(None)
28
29 # Visualizar convergencia
30 plt.figure(figsize=(12, 6))
31 for method, err in errors.items():
32     valid_n = [n for n, e in zip(n_values, err) if e is not None]
33     valid_err = [e for e in err if e is not None]
34     if valid_n:
35         plt.plot(valid_n, valid_err, 'o-', label=method)
36 plt.xlabel('Número de subintervalos (n)')
37 plt.ylabel('Error absoluto')
38 plt.title('Convergencia de m todos num ricos para      ^      sin(x) dx')
39 plt.legend()
40 plt.grid(True, alpha=0.3)
41 plt.yscale('log')
42 plt.show()
43
44 # Resultados con n=20
45 n = 20
46 left = riemann_left(f2, a, b, n)
47 right = riemann_right(f2, a, b, n)
48 mid = riemann_midpoint(f2, a, b, n)
49 trap = trapezoidal(f2, a, b, n)
50 simp = simpson(f2, a, b, n)
51
52 print("Resultados para      ^      sin(x) dx (n=20):")
53 print(f"    Valor exacto:      {exact:.6f}")
54 print(f"    Riemann izquierda:  {left:.6f} (error: {abs(left-exact):.6f})")
55 print(f"    Riemann derecha:    {right:.6f} (error: {abs(right-exact):.6f})")
56 print(f"    Punto medio:       {mid:.6f} (error: {abs(mid-exact):.6f})")
57 print(f"    Trapecio:          {trap:.6f} (error: {abs(trap-exact):.6f})")
58 print(f"    Simpson:           {simp:.6f} (error: {abs(simp-exact):.6f})")
59
60 # Visualizar
61 plot_riemann(f2, a, b, n, 'mid')
```

Listing 4: Código para el Ejemplo 2

6 Análisis de Resultados

6.1 Precisión de los Métodos

La tabla siguiente muestra el orden de convergencia de cada método:

Método	Orden	Observación
Riemann izquierda	$\mathcal{O}(h)$	Error lineal
Riemann derecha	$\mathcal{O}(h)$	Error lineal
Punto medio	$\mathcal{O}(h^2)$	Más preciso que los anteriores
Trapecio	$\mathcal{O}(h^2)$	Similar al punto medio
Simpson	$\mathcal{O}(h^4)$	Muy preciso para funciones suaves

Table 1: Orden de convergencia de los métodos numéricos

6.2 Propiedades de la Sumatoria Aplicadas

Las propiedades de la sumatoria permiten:

1. **Simplificación de sumas:** Para funciones polinómicas, podemos usar sumas notables para calcular límites exactos.
2. **Linealidad:** Permite descomponer integrales de funciones complejas.
3. **Implementación eficiente:** Las sumas se pueden vectorizar en Python usando NumPy.

7 Conclusión

La conexión entre sumatorias e integrales definidas es fundamental en el cálculo numérico. Las propiedades de la sumatoria no solo facilitan el cálculo analítico de límites, sino que también proporcionan la base para implementar algoritmos numéricos eficientes.

Python, con bibliotecas como NumPy, permite implementar estos métodos de manera sencilla y eficiente, aprovechando la vectorización para operaciones con sumatorias.

Los ejemplos presentados demuestran que:

- Los métodos de Riemann son conceptualmente simples pero menos precisos.
- La regla de Simpson ofrece la mejor precisión para funciones suaves.
- La elección del método depende del equilibrio entre precisión y costo computacional.

Referencias

1. Stewart, J. (2015). *Cálculo de una variable*. Cengage Learning.
2. Burden, R. L., & Faires, J. D. (2010). *Análisis Numérico*. Cengage Learning.
3. Press, W. H., et al. (2007). *Numerical Recipes*. Cambridge University Press.

```

x=0
s=1
for a in range(1,21):
    # print(a)
    s += 5
    s=s+a
    print(f"El valor de a: {a} y el valor de s después de incrementarlo a veces es: {s}")

```

```

El valor de a: 1 y el valor de s después de incrementarlo a veces es: 7
El valor de a: 2 y el valor de s después de incrementarlo a veces es: 14
El valor de a: 3 y el valor de s después de incrementarlo a veces es: 22
El valor de a: 4 y el valor de s después de incrementarlo a veces es: 31
El valor de a: 5 y el valor de s después de incrementarlo a veces es: 41
El valor de a: 6 y el valor de s después de incrementarlo a veces es: 52
El valor de a: 7 y el valor de s después de incrementarlo a veces es: 64
El valor de a: 8 y el valor de s después de incrementarlo a veces es: 77
El valor de a: 9 y el valor de s después de incrementarlo a veces es: 91
El valor de a: 10 y el valor de s después de incrementarlo a veces es: 106
El valor de a: 11 y el valor de s después de incrementarlo a veces es: 122
El valor de a: 12 y el valor de s después de incrementarlo a veces es: 139
El valor de a: 13 y el valor de s después de incrementarlo a veces es: 157
El valor de a: 14 y el valor de s después de incrementarlo a veces es: 176
El valor de a: 15 y el valor de s después de incrementarlo a veces es: 196
El valor de a: 16 y el valor de s después de incrementarlo a veces es: 217
El valor de a: 17 y el valor de s después de incrementarlo a veces es: 239
El valor de a: 18 y el valor de s después de incrementarlo a veces es: 262
El valor de a: 19 y el valor de s después de incrementarlo a veces es: 286
El valor de a: 20 y el valor de s después de incrementarlo a veces es: 311

```

```

print(f"Ingresar un valor real y entero positivo")
x=int(input("Ingresar el valor entero positivo: "))

for a in range(10,31):
    y=x+2
    z=y+1
    x += 1
    y1=y+x
    # print(f"Los valores son: {[x,a,y,z,x,y1]}")
    print(f"Los valores son: [x={x},a={a},y={y},z={z},y1={y1}]")

print(f"El valor de y1 es {y1}")

```

```

Ingresar un valor real y entero positivo
Ingresar el valor entero positivo: 2
Los valores son: [x=3,a=10,y=4,z=5,y1=7]
Los valores son: [x=4,a=11,y=5,z=6,y1=9]
Los valores son: [x=5,a=12,y=6,z=7,y1=11]
Los valores son: [x=6,a=13,y=7,z=8,y1=13]
Los valores son: [x=7,a=14,y=8,z=9,y1=15]
Los valores son: [x=8,a=15,y=9,z=10,y1=17]
Los valores son: [x=9,a=16,y=10,z=11,y1=19]
Los valores son: [x=10,a=17,y=11,z=12,y1=21]
Los valores son: [x=11,a=18,y=12,z=13,y1=23]
Los valores son: [x=12,a=19,y=13,z=14,y1=25]
Los valores son: [x=13,a=20,y=14,z=15,y1=27]
Los valores son: [x=14,a=21,y=15,z=16,y1=29]
Los valores son: [x=15,a=22,y=16,z=17,y1=31]
Los valores son: [x=16,a=23,y=17,z=18,y1=33]
Los valores son: [x=17,a=24,y=18,z=19,y1=35]
Los valores son: [x=18,a=25,y=19,z=20,y1=37]
Los valores son: [x=19,a=26,y=20,z=21,y1=39]

```

```
Los valores son: [x=20,a=27,y=21,z=22,y1=41]
Los valores son: [x=21,a=28,y=22,z=23,y1=43]
Los valores son: [x=22,a=29,y=23,z=24,y1=45]
Los valores son: [x=23,a=30,y=24,z=25,y1=47]
El valor de y1 es 47
```

```
n=0
for i in range(1,16):
    x=2*n+1
    n += 1
    print(f"El valor de n es: {n} y el valor de x es: {x}")
```

```
El valor de n es: 1 y el valor de x es: 1
El valor de n es: 2 y el valor de x es: 3
El valor de n es: 3 y el valor de x es: 5
El valor de n es: 4 y el valor de x es: 7
El valor de n es: 5 y el valor de x es: 9
El valor de n es: 6 y el valor de x es: 11
El valor de n es: 7 y el valor de x es: 13
El valor de n es: 8 y el valor de x es: 15
El valor de n es: 9 y el valor de x es: 17
El valor de n es: 10 y el valor de x es: 19
El valor de n es: 11 y el valor de x es: 21
El valor de n es: 12 y el valor de x es: 23
El valor de n es: 13 y el valor de x es: 25
El valor de n es: 14 y el valor de x es: 27
El valor de n es: 15 y el valor de x es: 29
```

```
import numpy as np

arreglo = np . array ([10 , 20 , 30 , 40 , 50])
suma = 0
for valor in arreglo :
    suma += valor

print (f"La suma del arreglo es :{suma}")
```

La suma del arreglo es :150

```
A=range(1,21)
arreglo=A
suma = 0
for valor in arreglo:
    suma += valor

print (f"La suma del arreglo es :{suma}")
```

La suma del arreglo es :210

```
matriz = [
    [1 , 2 , 3] ,
    [4 , 5 , 6] ,
    [7 , 8 , 9]
]

for fila in matriz :
    for elemento in fila :
        print ( elemento , end = " \n" )

print () # Salto de línea al terminar cada fila
```

1
2
3
4
5
6
7
8
9